

ALFIDO TECH INTERNSHIP PROGRAM

COHORT: MAY–JUNE 2026

- **Candidate Name:** Manish Kafle
- **Candidate ID:** BS/REG/120773
- **Domain:** Python Developer Intern
- **Project Task:** Task 2 – API Integration & JSON Handling

1. Project Objective

The goal of this project is to build a reliable, modular command-line interface (CLI) application in Python that securely communicates with an external REST API (OpenWeatherMap) over HTTPS. The application fetches real-time meteorological metrics for any user-specified global city, processes the raw nested response data, and returns a clean, structured JSON format.

Special emphasis was placed on secure credential configuration (decoupling API keys via environment variables), dependency isolation inside a virtual environment, strict data filtering, and exception handling boundaries for network dropouts and invalid user inputs.

2. Technical Stack Used

- **Core Language:** Python 3.x
- **Network Client:** Requests (HTTP library)
- **Environment Manager:** Python-Dotenv
- **Data Format:** Serialization via native json library
- **API Target:** OpenWeatherMap API Engine

3. Repository Links

- **Task Folder Repository Link:**
<https://github.com/CodedByManish/Alfido-python-internship/tree/main/task-02-api-weather>

4. Core Source Code (app.py)

```
import requests
import os
import json
from dotenv import load_dotenv

load_dotenv()

API_KEY = os.getenv("OPENWEATHER_API_KEY")
BASE_URL = "https://api.openweathermap.org/data/2.5/weather"

def get_weather(city):
    params = {"q": city, "appid": API_KEY, "units": "metric"}
    try:
        response = requests.get(BASE_URL, params=params, timeout=10)

        if response.status_code != 200:
            if response.status_code == 404:
                return "City not found"
            if response.status_code == 401:
                return "Invalid API key"
            return f"Error: {response.status_code}"

        data = response.json()

        result = {
            "city": data["name"],
            "temperature": data["main"]["temp"],
            "humidity": data["main"]["humidity"],
            "weather": data["weather"][0]["description"],
            "wind_speed": data["wind"]["speed"]
        }
        return result
    except requests.exceptions.Timeout:
        return "Request timeout"
    except requests.exceptions.ConnectionError:
        return "Connection error"
    except Exception as e:
        return f"Unexpected error: {str(e)}"

if __name__ == "__main__":
    city = input("Enter city name: ").strip()

    if not city:
        print("\nResult:")
        print(json.dumps({"error": "City name cannot be empty"}, indent=2))
    else:
        result = get_weather(city)
        print("\nResult:")

        if isinstance(result, dict):
            print(json.dumps(result, indent=2))
        else:
            print(json.dumps({"error": result}, indent=2))
```

5. Execution Screenshots & Explanations

Case A: Successful API Extraction & Filtered JSON Output

Description: The application was run within an isolated Python virtual environment (venv). The input parameter provided was Hyderabad. The script hit the API, verified status 200 OK, filtered down the raw response payload, serialized it using `json.dumps()`, and printed a structured schema.

```
api-json> python app.py
Enter city name: kathmandu

Result:
{
  "city": "Kathmandu",
  "temperature": 22.12,
  "humidity": 73,
  "weather": "broken clouds",
  "wind_speed": 1.03
}
```

Case B: Graceful API Error Handling (Invalid Input Boundary)

Description: To test the application's runtime stability, an invalid city value was provided. The application caught the incoming 404 status code anomaly cleanly, mapping it directly inside a standardized `{"error": "City not found"}` JSON formatting target instead of throwing an unhandled python stack trace or crashing.

```
api-json> python app.py
Enter city name: asdfghj

Result:
{
  "error": "City not found"
}
```

```
api-json> python app.py
Enter city name:

Result:
{
  "error": "City name cannot be empty"
}
```